Contents

Generative Adversarial Networks (GANs) 2

GANs for AES 11

GAN for RSA 13

How is this param, total params, trainable params, non-trainable params are created? What does they mean? 14

Effect of CNN hyperparameters on parameter counts? 17

Shift Rows: Incorrect (bcz 4×4 AES state is filled row by row) 18

Shift Rows: Correct (4×4 AES state is filled column by column) 20

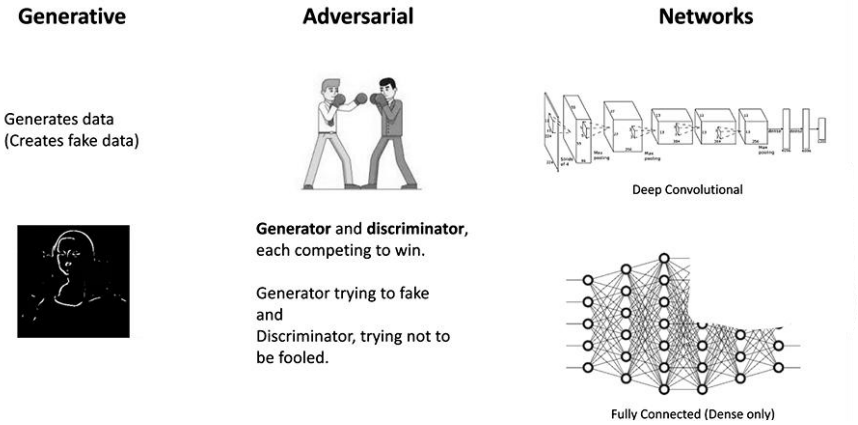
Generative Adversarial Networks (GANs)

A unsupervised learning model.

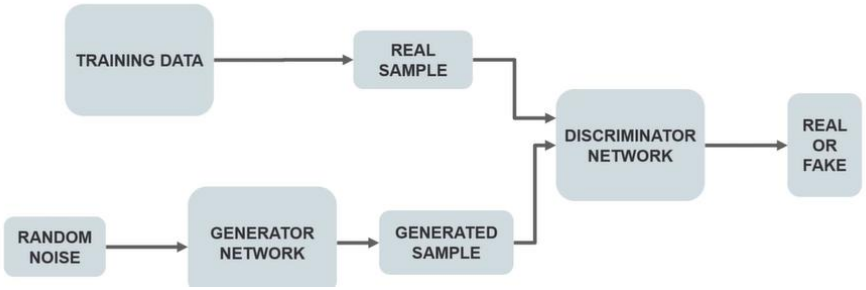
A GAN is a pair of neural networks — a **Generator (G)** and a **Discriminator (D)** — that are trained together.

The generator tries to produce realistic fake data from random noise; the discriminator tries to tell real data from the fake.

Over training, G learns to map a simple latent distribution (e.g., Gaussian) into samples that resemble the real data distribution.



WORKING:

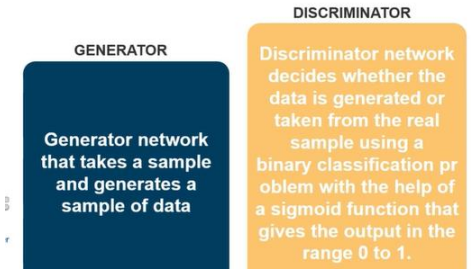


Task starts by random noise.

We are using 2 NN→

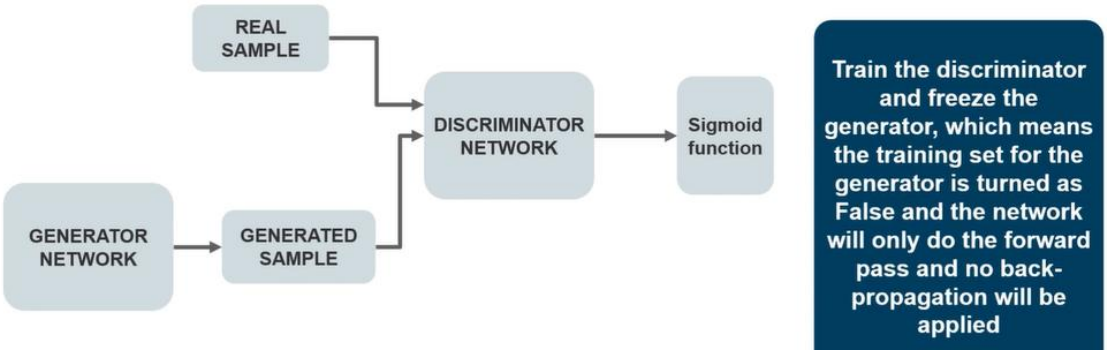
Generator (generates fake data) and Discriminator

These 2 NNs are trained simultaneously.

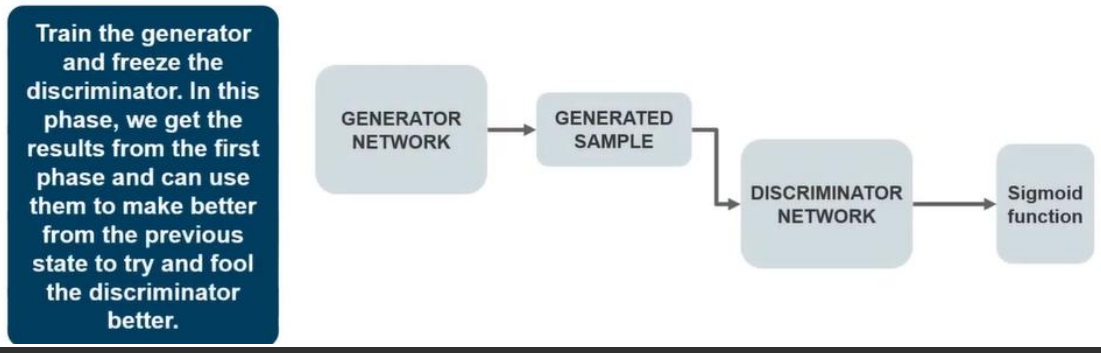


TRAINING happens in 2 phases→

1st Phase- we train discriminator and freeze generator. Training set for generator is false and network only so forward pass and no back propagation. Discriminator is trained with real data and checks if it can predicts them correctly and same with fake data to identify them as fake.



2nd Phase- Train generator and freeze discriminator. Get result from 1st phase and use them to make better from the previous state and try to fool discriminator better



How To Train A GAN?

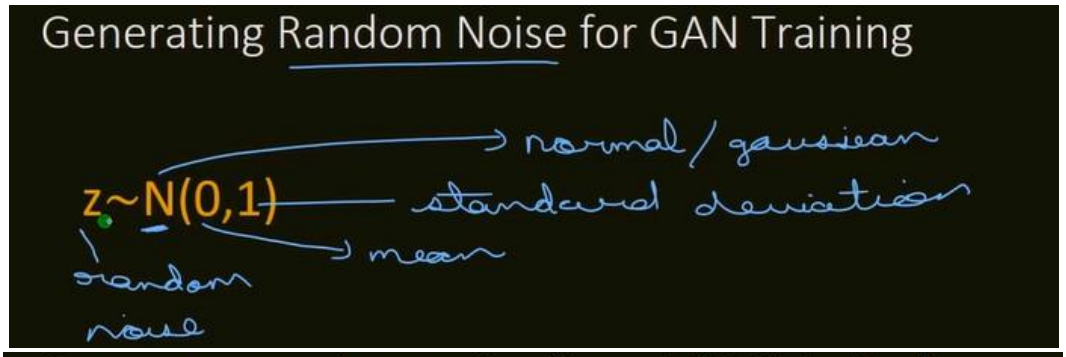
5 steps of GAN

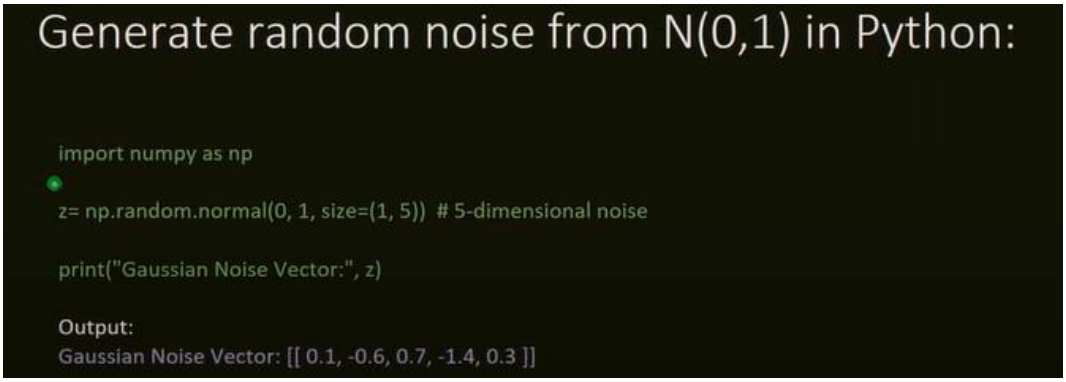
1. Define GAN architecture based on the application
2. Train discriminator to distinguish real vs fake data
3. Train the generator to fake data that can fool the discriminator
4. Continue discriminator and generator training for multiple epochs
5. Save generator model to create new, realistic fake data

NOTE: When training the discriminator, hold the generator values constant; and when training the generator, hold the discriminator constant. Each should train against a static adversary.

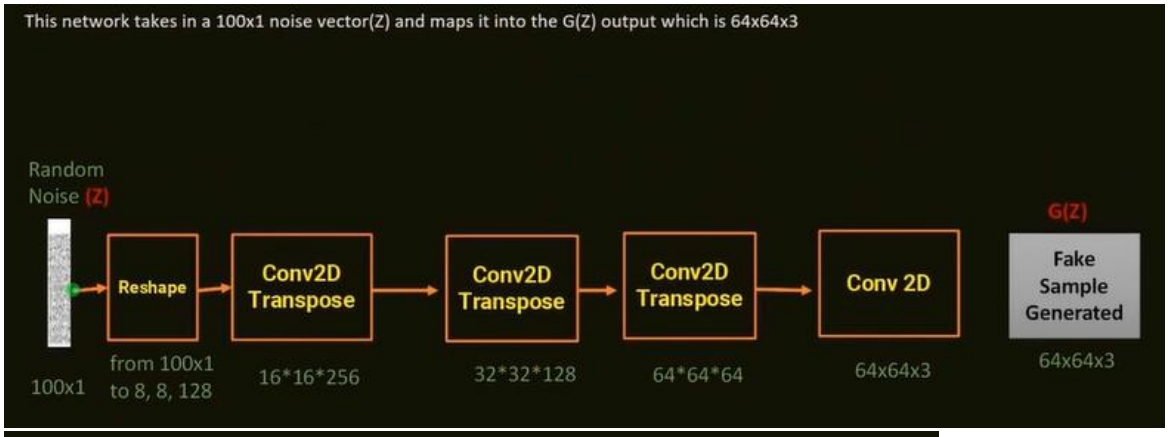
I/P to GAN- Random noise/ Random seed/ Latent Vector

RANDOM NOISE- sequence of random numbers. It can be 2D or 1D vector.





Generator- Generator network can be Convolutional Network or Dense Layers only.





h* width* channel

128= how may feature maps we want

We can do this using 2 layers→ dense + reshape

Formula for dense layer→ y= Wz+b where z= random noise

Example - Random noise with 3 values

Input Noise (z):

$$z = [0.1, -0.5, 0.8]$$

Weight Matrix (W):

$$W = \begin{bmatrix} 0.2 & -0.1 & 0.3 \\ 0.4 & 0.5 & -0.2 \\ -0.3 & 0.1 & 0.6 \\ 0.7 & -0.4 & 0.2 \end{bmatrix}$$

Bias Vector (b):

$$b = [0.1, -0.2, 0.3, 0.0]$$

Matrix Multiplication (Wz):

$$Wz = \begin{bmatrix} (0.2 \cdot 0.1) + (-0.1 \cdot -0.5) + (0.3 \cdot 0.8) \\ (0.4 \cdot 0.1) + (0.5 \cdot -0.5) + (-0.2 \cdot 0.8) \\ (-0.3 \cdot 0.1) + (0.1 \cdot -0.5) + (0.6 \cdot 0.8) \\ (0.7 \cdot 0.1) + (-0.4 \cdot -0.5) + (0.2 \cdot 0.8) \end{bmatrix}$$

$$Wz = \begin{bmatrix} 0.34 \\ -0.62 \\ 0.43 \\ 0.63 \end{bmatrix}$$

Bias Vector (b):

$$b = [0.1, -0.2, 0.3, 0.0]$$

Matrix Multiplication (Wz):

$$Wz = \begin{bmatrix} (0.2 \cdot 0.1) + (-0.1 \cdot -0.5) + (0.3 \cdot 0.8) \\ (0.4 \cdot 0.1) + (0.5 \cdot -0.5) + (-0.2 \cdot 0.8) \\ (-0.3 \cdot 0.1) + (0.1 \cdot -0.5) + (0.6 \cdot 0.8) \\ (0.7 \cdot 0.1) + (-0.4 \cdot -0.5) + (0.2 \cdot 0.8) \end{bmatrix}$$

$$Wz = \begin{bmatrix} 0.34 \\ -0.62 \\ 0.43 \\ 0.63 \end{bmatrix}$$

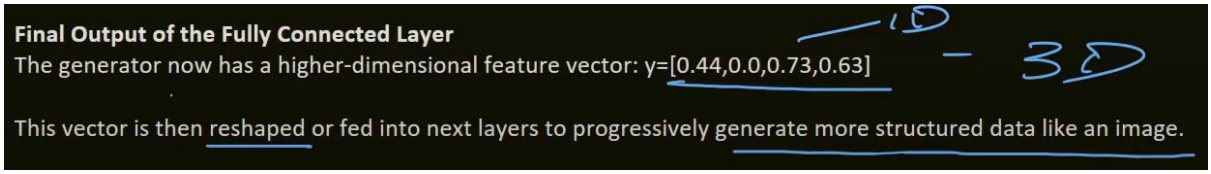
Add Bias ($Wz + b$):

$$y = Wz + b = \begin{bmatrix} 0.34 \\ -0.62 \\ 0.43 \\ 0.63 \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.2 \\ 0.3 \\ 0.0 \end{bmatrix} = \begin{bmatrix} 0.44 \\ -0.82 \\ 0.73 \\ 0.63 \end{bmatrix}$$

Apply Activation Function (ReLU): ReLU replaces all negative values with 0:

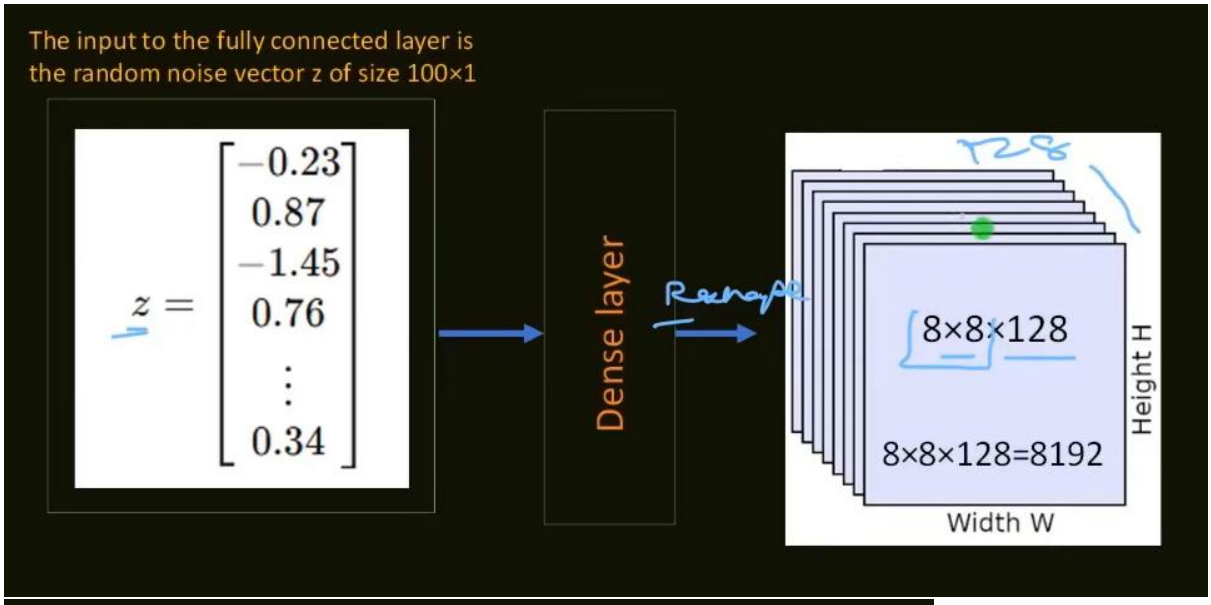
$$y = \text{ReLU}(y) = \begin{bmatrix} \max(0, 0.44) \\ \max(0, -0.82) \\ \max(0, 0.73) \\ \max(0, 0.63) \end{bmatrix} = \begin{bmatrix} 0.44 \\ 0.0 \\ 0.73 \\ 0.63 \end{bmatrix}$$

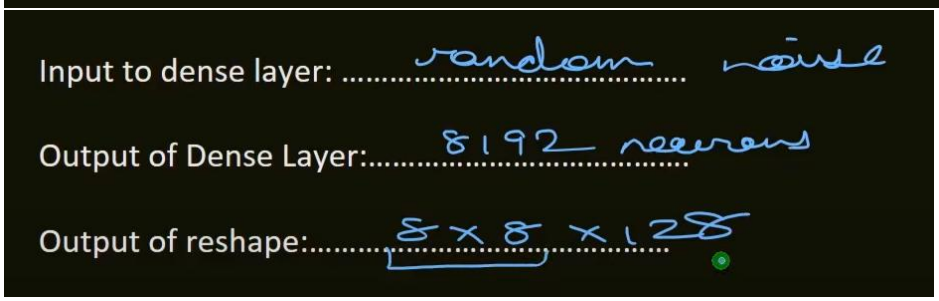
Dense layer is used bcz i/p random vector is 1d and we want to convert it into 2d. so, we are using dense to convert lower dimension to higher dimension.



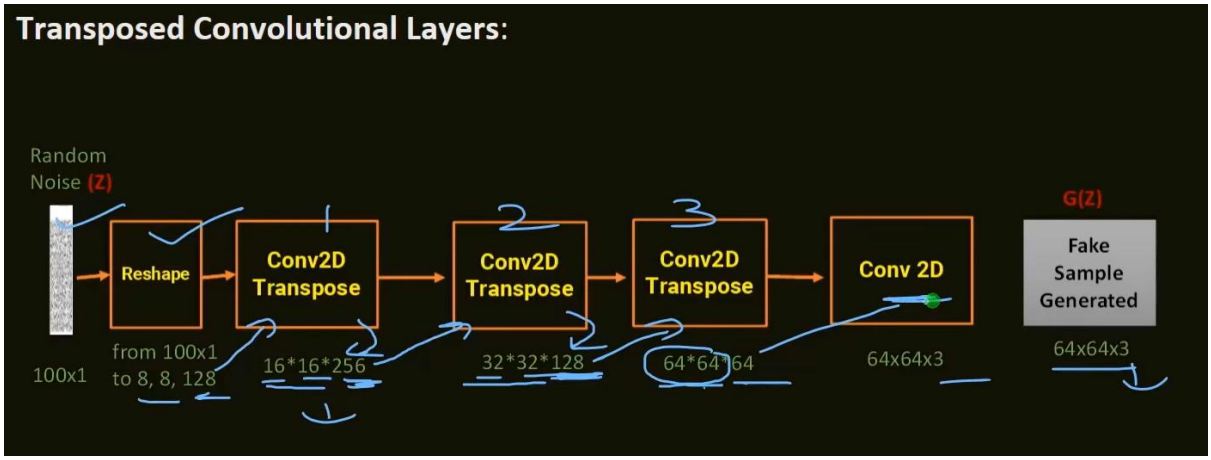
For img we need 3D densor i.e. height, width, no. of channels of img (for colored img we need 3 channels).

So, we are converting 1D vector into 3D using reshape.

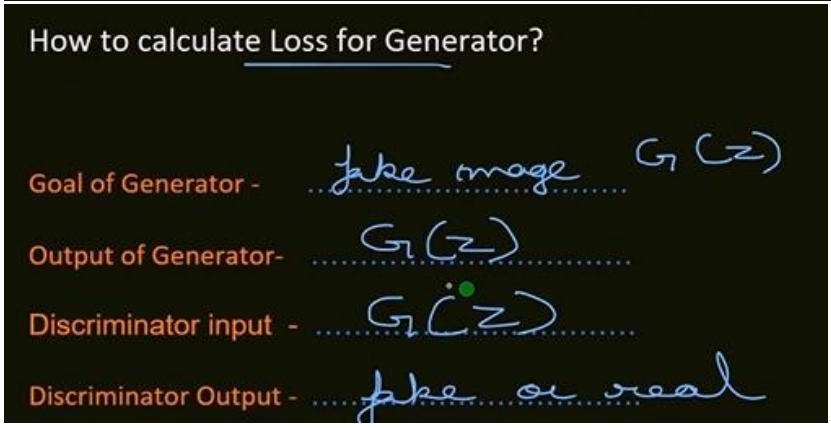




3 Conv2D transpose layers→ for upsampling. To increase height and width of img.

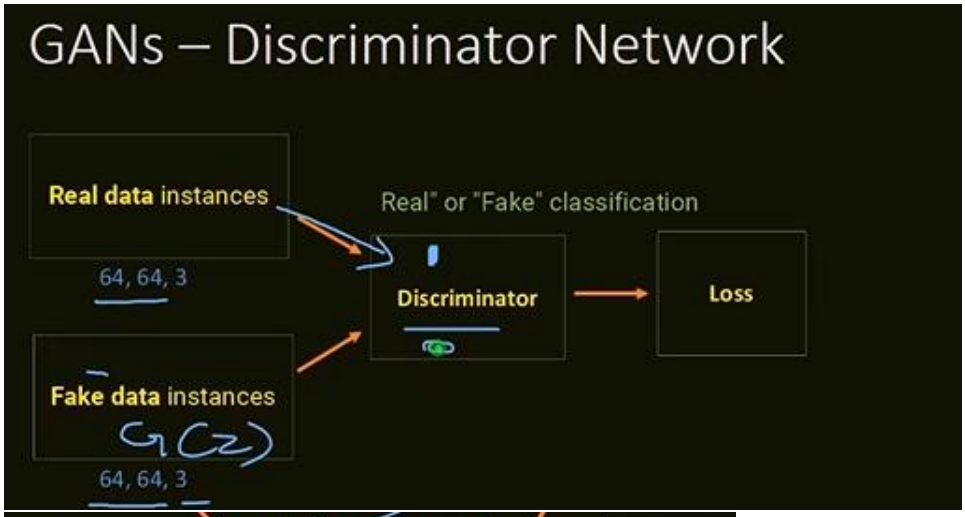


Layer	Resolution	Feature Maps
Dense + Reshape	8 x 8	128
Transposed Convolution 1	16 x 16	256
Transposed Convolution 2	32 x 32	128
Transposed Convolution 3	64 x 64	64
Output Layer	64 x 64	3

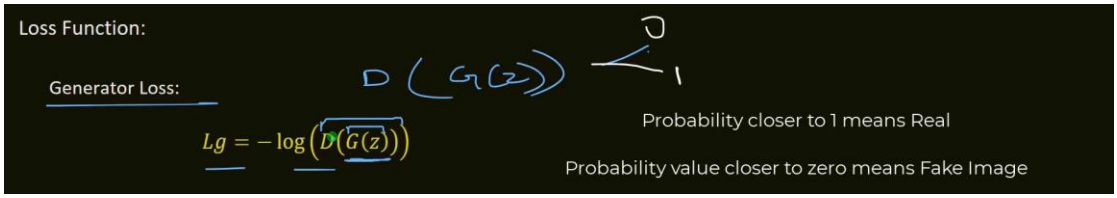


To calculate loss of generator we need to 1st understand the concept of discriminator.

Discriminator we will get value of loss of generator after getting feedback from generator.





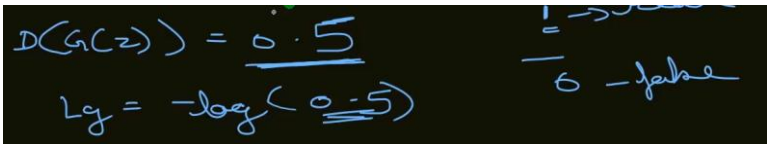


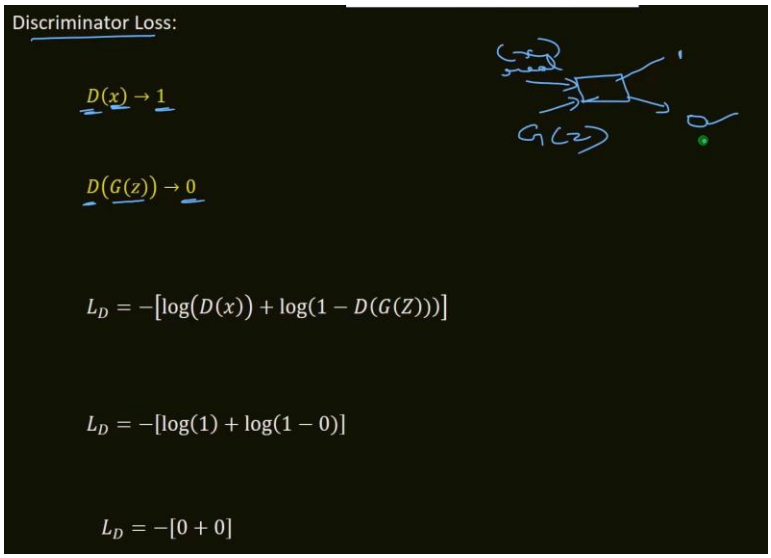
Lg= loss of generator, G(z)= fake data generated by the generator,D(G(z)) = fake data given to discriminator, discriminator willclassify and give probability value between 0-1.

Case 1: Generator is performing good-



Case 2: Generator needs inprovement and more training-





Loss Function

Loss functions that reflect the distance between the distribution of the data generated by the GAN and the distribution of the real data.

minimax loss function

$$E_x[log(D(x))] + E_z[log(1 - D(G(z)))]$$

The formula derives from the cross-entropy between the real and generated distributions.

it is designed to quantify the difference between two probability distributions

$$E_x[log(D(x))] + E_z[log(1 - D(G(z)))]$$

Discriminator output for Real Data

Discriminator output for fake Data

In this function:
D(x) is the discriminator's probability that real data instance x is real.
E_x is the expected value over all real data instances.
G(z) is the generator's output when given noise z.
D(G(z)) is the discriminator's estimate of the probability that a fake instance is real.
E_z is the expected value over all random inputs to the generator (in effect, the expected value over all generated fake instances G(z)).

So, loss of D =0. Means error=0 and 100% accuracy.

Example of poor performance Discriminator:

$$D(x) \rightarrow 0.5$$

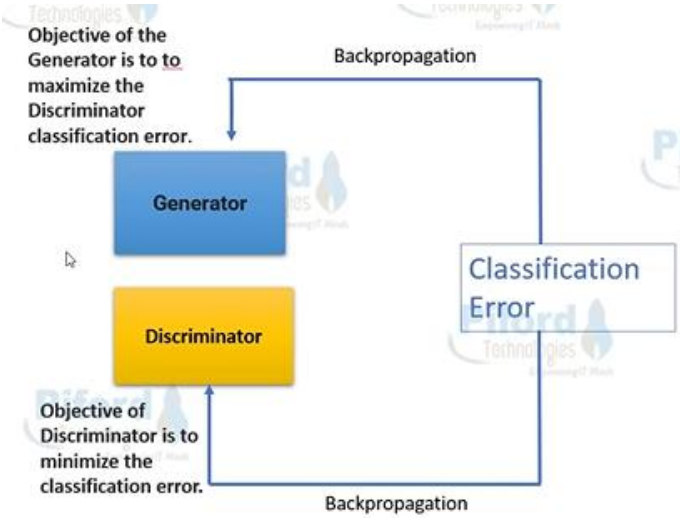
$$D(G(z)) \rightarrow 0.5$$

$$L_D = -[\log(D(x)) + \log(1 - D(G(Z)))]$$

$$L_D = -[\log(0.5) + \log(1 - 0.5)]$$

$$L_D = -[-0.693 - 0.693] = 1.386$$

GAN model is ready for output when discriminator performs poorly i.e discriminator give probabilitie values near to 1 for fake data. At this time we can stop training GAN.



Applications of GAN

- Generate fake data for augmenting other machine learning algorithms
- Generate faces (https://www.thispersondoesnotexist.com/)
- Image to image translation
- Text to image translation
- Super resolution

GANs for AES

A **Generative Adversarial Network (GAN)** consists of:

- **Generator (G):** Produces fake samples from noise/latent input.
- **Discriminator (D):** Judges whether a sample is real (from training data) or fake (from G).

How GANs Could Be Used for AES Key Recovery

Here are two possible setups:

A) GAN as a Key Generator (Unsupervised/Generative)

- **Generator (G):** Input = Concatenate(pt, ct) + random noise → output candidate key.
- **Discriminator (D):** Learns to distinguish between *real key–ciphertext pairs* and *fake ones from G*.
- Training forces G to output keys that "fool" D into thinking they are valid AES keys for those (pt, ct).

➞ This turns the task into **distribution learning**: G learns a distribution over possible keys conditioned on (pt, ct).
But: You still need labeled training (real keys), otherwise D cannot train.

B) GAN for Feature Extraction (Adversarial Regularization)

- Instead of GAN as a key predictor, use adversarial training to ensure:
  - The **latent representation** of (pt, ct) encodes key-related information.
  - A discriminator forces the encoder/representation to "hide" irrelevant noise.
- This is closer to **Adversarial Autoencoders (AAE)** or **InfoGAN**.

1. **Input**: Concatenated `(pt, ct)` → plaintext and ciphertext pair.
2. **Encoder/Feature Extractor (E)**:
   - Maps `(pt, ct)` → latent vector `z`.
   - Goal: `z` should **contain information about the key** while filtering out irrelevant noise.
3. **Predictor/Key Classifier (K)**:
   - Uses `z` → predicts key (or key bytes).
   - This is the **main supervised task**.
4. **Adversarial Discriminator (D)**:
   - Takes latent vector `z`.
   - Tries to detect if `z` still contains irrelevant/noisy info.
   - The Encoder **fights against D** by removing useless information.
   - This makes `z` more **key-informati↓nd noise-free**.

- 🔹 **Training Objective**
  - **Key Predictor Loss (L_key):** Ensures latent `z` has enough information to predict the key.
  - **Adversarial Loss (L_adv):** Encoder learns to "fool" the Discriminator → making `z` invariant to irrelevant features (noise, plaintext redundancy, etc.).
  - Final Loss = `L_total = L_key + λ * L_adv`
    - (λ is a balancing factor for regularization).
-

Why GANs *Might Not Be Ideal* Here

- You already know the exact mapping (AES = deterministic).
- You can generate infinite training data (pt, ct, key) → no need for GAN-based augmentation.
- The task is **supervised classification/regression** (predicting key bytes), not unsupervised generation.

GANs shine in *unlabeled, high-dimensional, distribution-matching tasks* (e.g., image synthesis, side-channel waveform generation).
But for **AES key recovery given known pairs**, a supervised CNN (or Transformer with attention/residuals) is typically:

- Easier to train.
- More stable (GANs are notoriously unstable).
- More accurate (since ground-truth labels exist).

When GANs *Would* Make Sense in Cryptanalysis

- **Side-channel attacks (SCA):** GANs can generate realistic power traces/EM traces to augment datasets.
- **Ciphertext-only attack simulations:** GANs can model plausible plaintext distributions.
- **Adversarial training:** Use GAN to learn robust features against adversarially perturbed ciphertexts.

In recent cryptanalysis papers, GANs are used more for **trace/data generation** rather than directly predicting AES keys.

GAN for RSA

GANs could be applied to side-channel attacks in a few ways:

- **Data Augmentation:** The profiling phase requires a large number of power traces to train the classifier effectively. If an attacker has a limited set of real traces, a GAN could be trained to generate new, synthetic power traces that are statistically similar to the real ones. This larger, augmented dataset could then be used to train a more robust DNN classifier.
- **Countermeasure Development:** From a defender's perspective, a GAN could be used to probe for weaknesses. The GAN's generator could try to create power traces that amplify any leaking information, while the discriminator would try to distinguish these "attack" traces from "safe" traces. This process could help developers create more secure hardware or software that doesn't produce distinguishable patterns.

1. Side-Channel Attack Enhancement (Most Practical Use)

- RSA implementations leak **timing, power, or EM traces**.
- Normally, deep learning models (CNN, RNN, Transformer) are used to classify side-channel traces to recover private key bits.
- **GANs can improve this by:**
    - **Data Augmentation:** GAN generates synthetic side-channel traces (timing/power) to train stronger models when only a small dataset is available.
    - **Adversarial Feature Extraction:** The encoder learns to extract key-related features, while a discriminator forces it to ignore irrelevant noise (like temperature or hardware fluctuations).
    - Helps in **generalizing across devices** (transfer learning).

Example: "GAN-generated EM traces" + classifier → predict RSA private exponent bits.

2. Adversarial Regularization for RSA Trace Classification

- Similar to AES example:
    - Input = Side-channel trace (or modular exponentiation steps).
    - Encoder = extracts latent features.
    - Key predictor = predicts RSA private exponent bits.
    - Discriminator = forces features to be **key-informative but noise-invariant**.

How is this param, total params, trainable params, non-trainable params are created? What does they mean?

3. How is this param, total params, trainable params, non-trainable params are created? What does they mean?



Model: "aes_byte_cnn"

Layer (type)	Output Shape	Param #	Connected to
pt (InputLayer)	(None, 16)	0	-
ct (InputLayer)	(None, 16)	0	-
embedding (Embedding)	(None, 16, 64)	16,384	pt[0][0]
embedding_1 (Embedding)	(None, 16, 64)	16,384	ct[0][0]
concatenate (Concatenate)	(None, 16, 128)	0	embedding[0][0], embedding_1[0][0]
conv1d (Conv1D)	(None, 16, 256)	98,560	concatenate[0][0]
batch_normalization (BatchNormalizatio…	(None, 16, 256)	1,024	conv1d[0][0]
conv1d_1 (Conv1D)	(None, 16, 256)	196,864	batch_normalizat…
dense (Dense)	(None, 16, 256)	65,792	conv1d_1[0][0]

Total params: 395,008 (1.51 MB)
Trainable params: 394,496 (1.50 MB)
Non-trainable params: 512 (2.00 KB)

- **Total params** = total number of scalar values stored by the model (all weights + biases + state variables).
- **Trainable params** = those scalars the optimizer will update by gradient descent (they appear in `model.trainable_weights`).
- **Non-trainable params** = stored parameters that are **not** updated by the optimizer (they appear in `model.non_trainable_weights`) — e.g. BatchNorm running mean/var, or weights from frozen layers.
  And:
  Total params = Trainable params + Non-trainable params.

Where these numbers come from (mechanics)

A Keras `Layer` can create variables via `self.add_weight(...)`. Each such variable has a shape (e.g. `(in_dim, out_dim)`), and the number of *scalars* in that variable = product of its shape dimensions. Keras collects:

- `layer.weights` (all variables)
- `layer.trainable_weights` (those with `trainable=True`)
- `layer.non_trainable_weights` (those with `trainable=False`)

When you call `model.summary()` Keras sums the sizes:

- `Total params` = sum(product(shape) for every `layer.weights`)
- `Trainable params` = sum(product(shape) for every `layer.trainable_weights`)
- `Non-trainable params` = `Total - Trainable`

If you set `layer.trainable = False` (e.g. to freeze for transfer learning), that layer's variables move from trainable to non-traina ↓

1. What "params" mean in the summary

- **Param # (per layer):** The number of trainable **weights + biases** that layer has.
- **Total params:** Sum of all parameters in the model.
- **Trainable params:** Parameters the optimizer will update during training (trainable=True).
- **Non-trainable params:** Parameters that exist in the model but are **not updated** during training (trainable=False).

🔹 **2. Layer-by-layer breakdown of your model**

Input layers (pt, ct)

- Just placeholders.
- **Params = 0** (they don't store weights).

Embedding layers

- Each embedding layer maps an integer (e.g., a byte 0–255) into a dense vector.
- Formula:

$$\text{params} = \text{vocab_size} \times \text{embedding_dim}$$

Your embedding layers:

- `(None, 16, 64)` output shape → sequence length = 16, embedding dim = 64.
- Assuming vocab size = 256 (byte values):

$$256 \times 64 = 16,384$$

Concatenate

- Just joins two embeddings → no weights.
- **Params = 0**

Conv1D

- Formula:

$$\text{params} = (\text{kernel_size} \times \text{in_channels} \times \text{filters}) + \text{filters}$$

  - first term = convolution weights
  - second term = biases

Your first Conv1D: `98,560 params`

- So:

$$98,560 = (k \times 128 \times 256) + 256$$

  Solving:

$$k \times 128 \times 256 = 98,304 \quad \Rightarrow \quad k = 3$$

So → **256 filters, kernel size 3, input channels 128.**

Your second Conv1D: `196,864 params`

- So:

$$196,864 = (k \times 256 \times 256) + 256$$

→ Again kernel size = 3, filters = 256, in_channels = 256.

BatchNormalization

- BN creates:
  - **gamma** (scale) → trainable
  - **beta** (shift) → trainable
  - **moving mean** (non-trainable)
  - **moving variance** (non-trainable)
- For each channel → 4 scalars.
- With 256 channels:

$$256 \times 4 = 1,024 \quad (512 \text{ trainable} + 512 \text{ non-trainable})$$

Dense

- Formula:

$$\text{params} = (\text{input_dim} \times \text{units}) + \text{units}$$

Yours: `65,792 params`

- If input dim = 256 and units = 256:

Plaintext (16 bytes) + ShiftRows output (16 bytes) = 32 bytes input.

Each byte is embedded (via Embedding layers).

CNN layers learn patterns between plaintext and ciphertext to predict the key bytes.

Dense at the end classifies into 256 possible byte values (per key byte).

Effect of CNN hyperparameters on parameter counts?

No. of filters

- Each filter has its own set of weights → **more filters = more params.**
- Conv params scale linearly with number of filters:

$$\text{params} \propto \text{filters}$$

Filter size (kernel size)

- Larger kernel → more weights per filter.
- Conv params scale linearly with kernel size:

$$\text{params} \propto \text{kernel_size}$$

Stride

- **Does not affect params.**
- Stride changes output size (feature map length), but weights stay the same.

Padding

- **Does not affect params.**
- Only changes how outputs align with edges.

Pooling

- Pooling (MaxPool, AvgPool) has **no weights → no params**.
- But it reduces output size, which affects the **next layer's input dim** (so downstream layers can have fewer params).

Shift Rows: Incorrect (bcz 4×4 AES state is filled row by row)

```
# ShiftRows: This spreads the data, adding diffusion.
def shift_rows(state_batch):
    state = state_batch.reshape(-1, 4, 4)
    for i in range(4):
        state[:, i] = np.roll(state[:, i], -i, axis=1)
    return state.reshape(-1, 16)
```

1) reshape(-1, 4, 4) — what -1 means

- state_batch is expected to be shape (batch_size, 16) or possibly (16,) for a single sample.
- reshape(-1, 4, 4) tells NumPy: make a 3-D array with shape (?, 4, 4) where ? is inferred automatically from the total number of elements.
    - If state_batch.shape == (B,16), then ? becomes B → result (B,4,4).
    - If state_batch.shape == (16,), ? becomes 1 → result (1,4,4).
- Using -1 is just a convenient, flexible way to accept any batch size. (You could replace -1 with state_batch.shape[0] when you know batch dimension in advance.)

2) How the data is arranged after reshape

State now has shape (batch, 4, 4). **Crucial point:** NumPy's reshape uses **row-major** (C) order by default. That means each row in the 4×4 matrix is a contiguous run of 4 bytes from the flattened 16-byte vector.

So for a single sample with state_batch = np.arange(16) (bytes 0..15), state = state_batch.reshape(4,4) becomes:

That is *row-major* layout

```
[[ 0,  1,  2,  3],   # row 0
 [ 4,  5,  6,  7],   # row 1
 [ 8,  9, 10, 11],   # row 2
 [12, 13, 14, 15]]   # row 3
```

3) The loop and np.roll

- state[:, i] selects the *i-th row* of every sample (shape (batch, 4) — batch × 4 columns).
- np.roll(..., -i, axis=1) rotates each row **left** by i positions (negative shift in np.roll = left rotation; positive = right rotation).
- AES ShiftRows rule:
    - row 0: shift left by 0
    - row 1: shift left by 1
    - row 2: shift left by 2
    - row 3: shift left by 3

So the code implements exactly that pattern (left shifts by row index). Using -i is correct here because np.roll interprets a negative shift as left rotation.

4) Flatten back

return state.reshape(-1, 16) converts back to flat 16-byte vectors per sample (row-major flatten), keeping the same batch count (the -1 again infers that).

Reshape with .reshape(-1, 4, 4):
→ Numpy uses **row-major order** by default.
→ So your 16 bytes [b0, b1, b2, …, b15] are laid out like this:

Row-major layout used by your code:

[b0 b1 b2 b3]

[b4 b5 b6 b7]

[b8 b9 b10 b11]

[b12 b13 b14 b15]

Then shift **row 0 by 0, row 1 by 1, etc.**

This matches **the idea of AES ShiftRows** (diffusion by cyclic row shifts),
but not the AES column-major standard mapping.

So, is implementation AES-standard?

- **No**, because AES expects the state to be in column-major order.
- **Yes**, in the sense that you are *consistently* treating the state in row-major format across your entire pipeline (AddRoundKey → SubBytes → ShiftRows).

That means:

- For **cryptanalysis / ML experiments**: your code is fine, because consistency matters more than strict AES byte ordering.
- For **actual AES compliance**: it's not correct, since the byte ordering doesn't match FIPS-197 (AES standard).

Shift Rows: Correct (4×4 AES state is filled column by column)

```
def shift_rows_aes(state_batch):
    # state_batch: shape (B,16) or (16,)
    state = state_batch.reshape(-1, 4, 4, order='F')   # column-major -> AES state layout
    for r in range(4):
        state[:, r, :] = np.roll(state[:, r, :], -r, axis=1)  # shift row r left by r
    return state.reshape(-1, 16, order='F')
```

This code: (1) fills the 4×4 matrix column-wise (AES order), (2) shifts rows left by r, (3) flattens back column-wise — which exactly matches AES ShiftRows.

1. AES-128 standard ShiftRows

- AES represents its **state** as a 4×4 byte matrix, but filled in **column-major order**.
- Example: If you have a flat 16-byte array [b0, b1, b2, …, b15], AES maps it as:

State (AES standard, column-major):

[b0 b4 b8 b12]

[b1 b5 b9 b13]

[b2 b6 b10 b14]

[b3 b7 b11 b15]

ShiftRows then shifts **rows** (not columns):

- Row 0 → no shift
- Row 1 → shift left by 1
- Row 2 → shift left by 2
- Row 3 → shift left by 3

2. order='F' in NumPy reshape

- By default, NumPy uses **row-major order** (order='C', like C language).
- But AES defines its state in **column-major order** (like Fortran).
- order='F' = Fortran-style memory layout → fill **columns first** instead of rows.

3. state[:, r, :] = np.roll(state[:, r, :], -r, axis=1)

- state has shape (B, 4, 4) → (batch, row, column).
- state[:, r, :] selects **row r across the batch** (shape (B, 4)).

- np.roll(..., -r, axis=1):
    - Shifts that row's elements left by r positions (cyclic shift).
    - -r means left shift (positive r would be right shift).
    - axis=1 = shift along the **columns** (AES rows).

So for each row:

- Row 0: shift by 0 (no change).
- Row 1: shift by 1 (cyclic left by 1).
- Row 2: shift by 2 (cyclic left by 2).
- Row 3: shift by 3 (cyclic left by 3).

4. Final reshape back

- Converts (B, 4, 4) state back to (B, 16) flat array.
- Using order='F' ensures the flattening is **column-major**, i.e. the same mapping AES uses.